

File: decsion_tree/kfold_cv.py

Purpose: Decision Tree Model for classification or regression tasks

```
import random
import numpy as np
```

=== Imports ===

```
from sklearn.datasets import load_iris
# Simple accuracy
```

=== Functions ===

Logic Focus: Defined task specifically for 'accuracy_score'

```
def accuracy_score(y_true, y_pred):
    if len(y_true) == 0:
        return 0.0
    return sum(1 for a, b in zip(y_true, y_pred) if a == b) / len(y_true)
# Decision tree functions (direct implementation for CV)
```

=== Imports ===

```
from collections import Counter
import math
```

=== Functions ===

Logic Focus: Defined task specifically for 'entropy'

```
def entropy(labels):
    n = len(labels)
    if n == 0:
        return 0.0
    counts = Counter(labels)
    ent = 0.0
    for c in counts.values():
        p = c / n
        ent -= p * math.log2(p) if p > 0 else 0
    return ent
```

Logic Focus: Defined task specifically for 'gini_index'

```
def gini_index(groups, classes):
    n_instances = sum([len(g) for g in groups])
    if n_instances == 0:
        return 0.0
    gini = 0.0
    for group in groups:
        size = len(group)
        if size == 0:
            continue
        score = 0.0
        counts = Counter(group)
        for class_val in classes:
            p = counts.get(class_val, 0) / size
            score += p * p
        gini += (1.0 - score) * (size / n_instances)
    return gini
```

Logic Focus: Defined task specifically for 'best_split'

```
def best_split(X, y, criterion='entropy'):
    n_samples, n_features = X.shape
    best = {'index': None, 'threshold': None, 'score': -float('inf'), 'groups': None}
    classes = list(set(y))
    for feature_idx in range(n_features):
        values = sorted(set(X[:, feature_idx]))
        thresholds = [(values[i] + values[i+1]) / 2.0 for i in range(len(values)-1)]
        for thr in thresholds:
            left_idx = [i for i, row in enumerate(X) if row[feature_idx] <= thr]
            right_idx = [i for i, row in enumerate(X) if row[feature_idx] > thr]
            left_labels = [y[i] for i in left_idx]
            right_labels = [y[i] for i in right_idx]
            if criterion == 'entropy':
                parent_ent = entropy(y)
                n = len(y)
                w_ent = 0.0
                for group in (left_labels, right_labels):
                    w_ent += (len(group)/n) * entropy(group) if len(group) > 0 else 0
                info_gain = parent_ent - w_ent
                score = info_gain
            else:
                gini = gini_index([left_labels, right_labels], classes)
                score = -gini
            if score > best['score'] and len(left_labels) > 0 and len(right_labels) > 0:
                best = {'index': feature_idx, 'threshold': thr, 'score': score, 'groups': (left_idx,
right_idx)}
    return best
```

Logic Focus: Defined task specifically for 'to_terminal'

```
def to_terminal(y):
    return Counter(y).most_common(1)[0][0]
```

Logic Focus: Defined task specifically for 'split'

```
def split(node, X, y, max_depth, min_size, depth, criterion='entropy'):
    left_idx, right_idx = node['groups']
    del(node['groups'])
    if not left_idx or not right_idx:
        node['left'] = node['right'] = to_terminal([y[i] for i in left_idx + right_idx])
        return
    if depth >= max_depth:
        node['left'] = to_terminal([y[i] for i in left_idx])
        node['right'] = to_terminal([y[i] for i in right_idx])
        return
    if len(left_idx) <= min_size:
        node['left'] = to_terminal([y[i] for i in left_idx])
    else:
        X_left = X[left_idx]
        y_left = [y[i] for i in left_idx]
        node_left = best_split(X_left, y_left, criterion=criterion)
        node['left'] = node_left
        split(node['left'], X_left, y_left, max_depth, min_size, depth+1, criterion)
    if len(right_idx) <= min_size:
        node['right'] = to_terminal([y[i] for i in right_idx])
    else:
        X_right = X[right_idx]
        y_right = [y[i] for i in right_idx]
        node_right = best_split(X_right, y_right, criterion=criterion)
        node['right'] = node_right
        split(node['right'], X_right, y_right, max_depth, min_size, depth+1, criterion)
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

Logic Focus: Defined task specifically for 'predict_row'

```
def predict_row(node, row):
    if isinstance(node, dict) and node.get('index') is not None:
        if row[node['index']] <= node['threshold']:
            return predict_row(node['left'], row)
        else:
            return predict_row(node['right'], row)
    else:
        return node
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

Logic Focus: Defined task specifically for 'predict'

```
def predict(node, X):
    return [predict_row(node, row) for row in X]
```

K-fold CV

Logic Focus: Defined task specifically for 'kfold_cv_iris'

```
def kfold_cv_iris(k=5, max_depth=4, min_size=1, criterion='entropy', seed=1):
    data = load_iris()
    X = data.data
    y = list(data.target)
    n = len(X)
    idx = list(range(n))
    random.seed(seed)
    random.shuffle(idx)
    folds = []
    fold_size = n // k
    for i in range(k):
        start = i * fold_size
        if i == k-1:
            fold_idx = idx[start:]
        else:
            fold_idx = idx[start:start+fold_size]
        folds.append(fold_idx)
    scores = []
    for i in range(k):
        test_idx = folds[i]
        train_idx = [j for f in range(k) if f != i for j in folds[f]]
        X_train = X[train_idx]
        y_train = [y[j] for j in train_idx]
        X_test = X[test_idx]
        y_test = [y[j] for j in test_idx]
        root = best_split(X_train, y_train, criterion=criterion)
        split(root, X_train, y_train, max_depth, min_size, depth=1, criterion=criterion)
        y_pred = predict(root, X_test)
        sc = accuracy_score(y_test, y_pred)
        scores.append(sc)
        print(f'Fold {i+1}: accuracy={sc:.4f}')
    print(f'Average accuracy: {np.mean(scores):.4f} (std {np.std(scores):.4f})')
    return scores
```

```
if __name__ == '__main__':
    kfold_cv_iris(k=5, max_depth=4, min_size=1, criterion='entropy', seed=2)
```